# Практическая работа №5

## **Тема:** Моделирование. Архитектура «клиент–сервер»

**Цель работы**

Сформировать целостную модель клиент–серверной системы на выбранной теме: поведение, контракт взаимодействия, топологию и политику надёжности.

**Задание**

**1. Сформулируйте сюжет и два сценария.**

Продолжайте работать по утверждённой тематике. Коротко опишите два пользовательских сценария: **основной** (на нём ведёте все артефакты) и **расширяющий** (показывает масштабирование/вариативность). Укажите желаемые свойства качества в человеческих метриках: «p95 ответа ≤ 300 мс», «доставка предупреждения ≤ 60 с», «3 шага до результата».

**2. Покажите поведение глазами клиента и сервера.**

На одной странице дайте **C4 Dynamic Diagram** - https://c4model.com/diagrams/dynamic (или эквивалентную схему BPMN 2.0 Choreography/BPMN 2.0 Collaboration/Service Blueprint) для клиента и для сервера: акторы, границы ответственности, что считается «успехом». Добавьте 2–3 абзаца пояснений словами. Цель — чтобы читатель за минуту понял, кто чего ждёт от кого.ы

**3. Задайте контракт взаимодействия (без реализации).**

Оформите **интерфейс как артефакт**. Допустимо два варианта:

- Таблица «Ресурс → Операции → Статусы ответа → Ошибки → Требуемые заголовки/аутентификация» с краткими примерами полезных нагрузок **в виде схем** (JSON-схема/описание полей, а не исполняемый код).
- Либо файл **OpenAPI (YAML/JSON)** как спецификация протокола.
  В обоих случаях пометьте обязательные/необязательные поля, инварианты и версии контракта.

**4. Смоделируйте надёжность взаимодействий.**

Нарисуйте **диаграмму последовательностей** для основного сценария с разветвлениями alt/opt: нормальный ход, таймаут, повтор, сетевой сбой. Отдельно отразите **политику идемпотентности** (где и как используется ключ; что возвращает сервер при повторной подаче того же запроса) и **правила ретраев** (по умолчанию безопасные операции чтения; для изменяющих — только с ключом). Добавьте **диаграмму состояний** для жизненного цикла запроса/ресурса.

**5. Проясните транспорт и ограничения канала.**

Краткой заметкой и схемой покажите, что ваш HTTP работает поверх надёжного транспорта: где появляется рукопожатие, как используется keep-alive, где границы по числу одновременных соединений, что происходит при потере пакетов.

**6. Отобразите топологию развёртывания.**

Сделайте **диаграмму Deployment**. На схеме должны быть клиент(ы), сервер(ы), БД/очереди/внешние сервисы, сети/границы отказов и места горизонтального

масштабирования. Подпишите, где применяются политики из п. 4 (таймауты/ретраи/идемпотентность), и какие SLO/SLI относятся к каждой связи.

**7. Зафиксируйте наблюдаемость и безопасность как схему.**

Схема потоков **логов/метрик/трейсинга**: откуда берутся корреляционные идентификаторы, какие события журналируются, какие базовые метрики нужны (например, счётчик запросов, ошибки, p95). Отдельно — схематично **аутентификация/авторизация** (какой провайдер, где проверка токена, какие роли влияют на сценарий).

**8. Смоделируйте структуру данных и взаимодействие с БД**

Определите основные сущности предметной области, необходимые для реализации выбранного сценария. Постройте ER-диаграмму, отражающую сущности, их атрибуты и взаимосвязи. Укажите первичные и внешние ключи, ограничения целостности данных.

Для основных операций API определите, какие действия выполняются над базой данных (создание, получение, изменение, удаление записей). Отразите последовательность взаимодействия клиента, серверной части и базы данных.

**9. Оцените предполагаемую нагрузку на систему**

Определите предполагаемое количество пользователей, частоту обращений к основным операциям API и возможные ограничения серверных ресурсов. На основании сформулированных предположений выделите потенциальные узкие места системы.

Предложите способы повышения производительности: оптимизацию обращений к БД, кэширование, асинхронную обработку или горизонтальное масштабирование. Обоснуйте необходимость выбранных решений.